06 — SAP Chatbot Architecture

What it is: the capstone Python template — one config object + one class that combines LLM (02) + SQL query (03) + Vector/RAG (04-05) + memory into four selectable modes.

Config-Driven Design

```python
@dataclass
class ChatbotConfig:
    llm_model: str = "gpt-4o-mini"
    temperature: float = 0.1
    memory_window: int = 10          # 0 = stateless

    enable_rag: bool = False         # turns on HANA vector search
    vector_table: str = "VECTOR_STORE"
    rag_top_k: int = 5

    enable_sql: bool = False         # turns on NL-to-SQL
    sql_tables: list = field(default_factory=list)

    system_prompt: str = "You are a helpful SAP AI assistant."

    def mode(self) -> str:
        parts = []
        if self.enable_rag: parts.append("RAG")
        if self.enable_sql: parts.append("SQL")
        return "+".join(parts) if parts else "GENERAL"
```

Four modes from the same class, just by flipping flags:

Mode	enable_rag	enable_sql	Behaviour
GENERAL	✗	✗	plain LLM chat + memory
RAG	✓	✗	document Q&A with citations
SQL	✗	✓	NL → SQL → DataFrame answers
FULL	✓	✓	both, combined in one prompt

Core Class Shape

```python
class SAPChatbot:
    def __init__(self, config: ChatbotConfig, conn=None):
        if (config.enable_rag or config.enable_sql) and conn is None:
            raise ValueError("HANA connection required for RAG/SQL mode.")
        self._llm      = build_llm(config)
        self._embedder = build_embedding_model(config) if config.enable_rag else None
        self._history  = []                    # HumanMessage / AIMessage list

    def chat(self, message: str) -> dict:
        # 1. optionally retrieve RAG context and/or run SQL
        # 2. build final prompt via prompt_builder.build_prompt(...)
        # 3. call self._llm with history + new message
        # 4. append to self._history, return {"answer","sources","sql_used","mode","turn"}
        ...

    def reset(self):
        self._history = []
```

Key Points

- **One config object drives everything** — no other file needs editing to change behaviour; this is the dataclass/strategy pattern applied to chatbot design.

- Constructor **fails fast**: if RAG or SQL mode is requested without a `conn`, it raises immediately rather than failing later mid-conversation.
- The response dict is **structured** (`answer`, `sources`, `sql_used`, `mode`, `turn`) so a UI layer (Streamlit, or a UI5 app — see sheet 10) can render citations/SQL without string parsing.
- `cli_runner.py` (interactive terminal loop) and a Streamlit `app.py` are two thin front-ends over the exact same `SAPChatbot` class — the architecture is UI-agnostic.

Interview Q&A

Q: Why use a dataclass for configuration instead of passing many function arguments? A: Centralizes every tunable setting in one typed, self-documenting object; adding a new setting doesn't change every function signature that touches the bot.

Q: How does FULL mode (RAG+SQL) decide which source to use for a given question? A: The prompt builder includes both retrieved document context and a note that SQL tools are available; in the templates here it's the LLM's judgment guided by the system prompt — a more advanced version would use explicit intent classification or LangGraph routing (see your LangGraph practice notebooks) before choosing a path.

Q: What would you change to make this production-grade instead of a template? A: Per-user session storage instead of one in-memory `_history` list, rate limiting, structured logging (without raw PII), and moving from the Gen AI Hub proxy to the Orchestration Service for built-in content filtering/masking (sheets 07-08).